

PRACTICAL4

Create a scenario where a child process becomes a zombie process.
Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    int i, status;
```

```
    // Create multiple child processes
```

```
    for (i = 0; i < 3; i++) {
```

```
        pid = fork();
```

```
        if (pid == 0) {
```

```
            printf("Child %d (PID=%d) executing...\n", i+1, getpid());
```

```
            sleep(2); // simulate work
```

```
            exit(0);
```

```
        }
```

```
    }
```

```
    // Parent synchronizes children
```

```
    for (i = 0; i < 3; i++) {
```

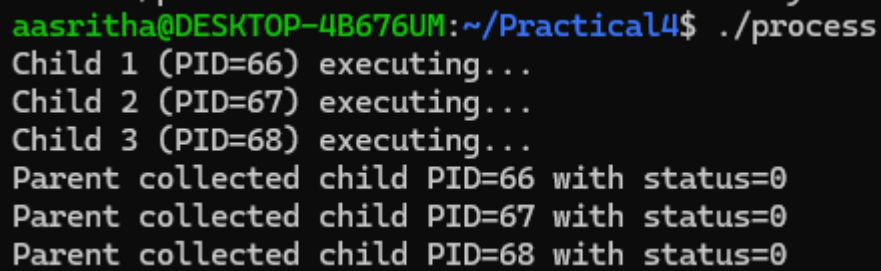
```

        pid_t wpid = wait(&status); // try waitpid(-1, &status, 0) here

        printf("Parent collected child PID=%d with status=%d\n", wpid,
WEXITSTATUS(status));
    }

    return 0;
}

```



```

aasritha@DESKTOP-4B676UM:~/Practical4$ ./process
Child 1 (PID=66) executing...
Child 2 (PID=67) executing...
Child 3 (PID=68) executing...
Parent collected child PID=66 with status=0
Parent collected child PID=67 with status=0
Parent collected child PID=68 with status=0

```

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

```

```

int main() {

    pid_t pid = fork();

    if (pid == 0) {

        printf("Child (PID=%d) exiting...\n", getpid());

        exit(0);

    } else {

        printf("Parent (PID=%d) sleeping...\n", getpid());

        sleep(20); // parent ignores child

    }
}

```

```
    return 0;
}
```

```
aasritha@DESKTOP-4B676UM:~/Practical4$ nano zombie.c
aasritha@DESKTOP-4B676UM:~/Practical4$ gcc zombie.c -o zombie
aasritha@DESKTOP-4B676UM:~/Practical4$ ./zombie
Parent (PID=75) sleeping...
Child (PID=76) exiting...
```

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>
```

```
int main() {

    pid_t pid = fork();

    if (pid == 0) {

        printf("Child (PID=%d) exiting...\n", getpid());

        exit(0);

    } else {

        int status;

        waitpid(pid, &status, 0); // reaps child immediately

        printf("Parent collected child PID=%d\n", pid);

        sleep(20); // no zombie now

    }

    return 0;
}
```

```
aasritha@DESKTOP-4B676UM:~/Practical4$ nano sync.c
aasritha@DESKTOP-4B676UM:~/Practical4$ gcc sync.c -o sync
aasritha@DESKTOP-4B676UM:~/Practical4$ ./sync
Child (PID=93) exiting...
Parent collected child PID=93
|
```
